Distributed Energy Monitoring Platform

Distributed Systems Course Project

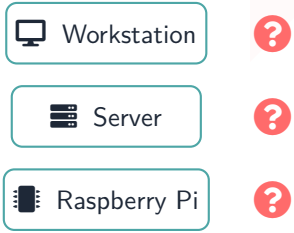
Youness Anouar - Mouad El Ansari - Jaafar Yeffou

UM6P College Of Computing | April 7, 2026

The Challenge

The Problem

Labs and organizations operate many workstations, servers, and Raspberry Pis, but energy usage is **fragmented** and often **invisible** in real time.



- No unified view of live power consumption
- No historical data for optimization
- Local-only monitoring creates data silos
- Manual sampling is too slow for operations

Goal: build one distributed system that scales from 10 to 10,000 nodes without redesigning the API or database.

What KPIs Do We Monitor?

Node-Level KPIs

- **Power (W)**: real or estimated node consumption
- **CPU usage**: utilization per machine
- **Memory usage**: RAM pressure over time
- **Temperature**: thermal behavior when available

Health KPI

Node status: ONLINE / STALE / OFFLINE

Application-Level KPIs

- **Per-app power**: who consumes the most energy
- **Per-app CPU share**: workload attribution
- **Top consumers**: ranking of active processes

Why These KPIs Matter

- Detect overloaded or unhealthy nodes
- Identify power-hungry applications

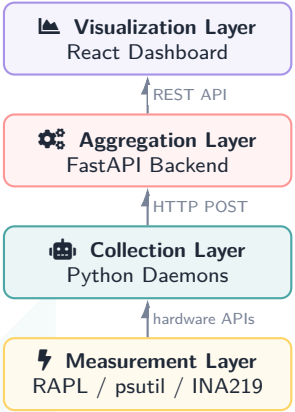
Why This Is a Distributed Systems Problem

- **Heterogeneous infrastructure**: Linux, Windows, and RPi expose different measurement interfaces.
- **Network unreliability**: nodes may disconnect unpredictably.
- **Temporal consistency**: distributed sources send at different rates.
- **Scalability**: aggregate large metric volumes in near real time.
- **Fault detection**: distinguish network jitter from actual failure.

Design Philosophy

- Autonomous daemons
- Eventual consistency
- Adaptive fault detection
- Time-series storage

Architecture at 10,000 Feet



- **Push architecture**: nodes send data proactively.
- **Stateless backend**: easy horizontal replication.
- **ClickHouse**: fast real-time and historical queries.

Containerized Deployment

The project stack is packaged in **Docker**, which makes backend services reproducible, portable, and easier to deploy across environments.

Distributed Systems Principles Applied

Fault Tolerance

- File-backed retry buffer
- Exponential backoff
- Daemon collects while backend is down

Eventual Consistency

- Nodes send independently
- Out-of-order arrival is acceptable
- Storage reconciles time-series state

Adaptive Health Checking

- Learned timeout per node
- ONLINE $\rightarrow$ STALE $\rightarrow$ OFFLINE
- 4σ threshold for low false positives

Horizontal Scalability

- Add nodes by deploying more daemons
- Add API replicas without coordination
- Same schema and query interface

Smart Solution 1: Heterogeneous Data Collection

Unified Interface, Different Backends

- **Linux**: Intel RAPL for direct hardware readings
- **Windows**: psutil-based estimation
- **Raspberry Pi**: INA219 sensor over I^2C
- **Fallback**: estimate from CPU when direct measurement is missing

Adapter pattern keeps one daemon API while supporting multiple platforms.

Platform	Method	Fidelity
Linux	RAPL	High
Windows	CPU estimate	Medium
RPi	INA219	High
Generic	Fallback	Medium

Schema Tags

`node_type`

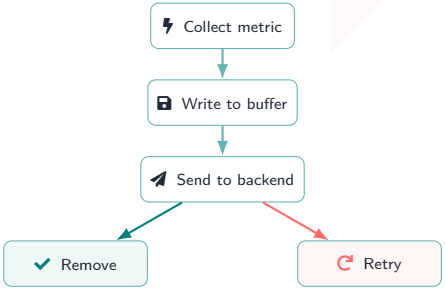
`measurement_method`

Smart Solution 2: Reliable Ingestion

Retry Buffer

1. Collect metrics every 5–10 seconds
2. Persist locally first
3. Try HTTP POST to backend
4. On failure, keep data in local buffer
5. Retry with exponential backoff

No data loss during temporary outages and no dependency on constant connectivity.



Smart Solution 3: Adaptive Heartbeat Detection

k-Sigma Timeout

$$\text{timeout} = \mu_{\text{interval}} + 4 \times \sigma_{\text{interval}}$$

- Fast nodes get short timeouts
- Slow nodes get relaxed timeouts
- STALE state absorbs temporary jitter
- OFFLINE declared only after stronger evidence

Mimics heartbeat-based fault detection used in production distributed systems.

Node	Pattern	Timeout
Node-1	10s, stable	10.6s
Node-2	60s, slower	68s
Node-3	variable	55s

Health States

- **ONLINE**: age $<$ timeout
- **STALE**: timeout $\leq$ age $<$ 2×timeout
- **OFFLINE**: age $\geq$ 2×timeout

Smart Solution 4: Aggregation at Scale

Problem

100 nodes, multiple apps, and several KPIs create thousands of time series. Raw rendering will **overwhelm the browser**.

Solution

- ClickHouse stores raw metrics in columnar format
- Materialized views precompute hourly/daily summaries
- Dashboard selects raw or aggregated reads by range
- Result sets stay under ∼20K points per chart

Range	Level	Points
1 day	raw	8.6K–17.2K
7 days	1 minute	10.1K
30 days	5 minute	8.6K
1 year	hourly	8.8K

Trade storage space for consistently fast queries and stable rendering.

Smart Solution 5: Stateless API for Unreliable Clients

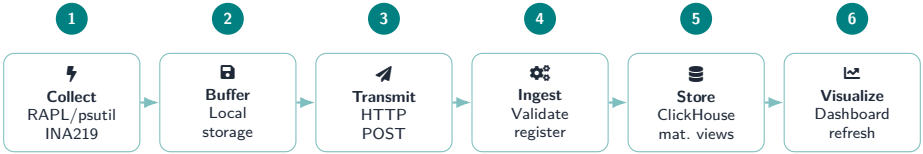
- **Idempotent auto-registration**: create a node only if it does not already exist.
- **Graceful degradation**: estimate power when a client cannot provide a direct reading.
- **Async heartbeat update**: health tracking does not block ingestion response time.
- **Safe retries**: repeated metric submissions do not corrupt shared state.

Why It Matters

A stateless ingestion layer scales **horizontally** and stays robust when clients time-out, disconnect, or resend requests.

Measured ingestion path: $\sim$2 ms inside the backend before data is available for query.

End-to-End Data Flow



Typical End-to-End Latency

Around **one second** from collection on the node to visibility on the dashboard.

Paper Benchmarks: Figure 3 and Figure 5

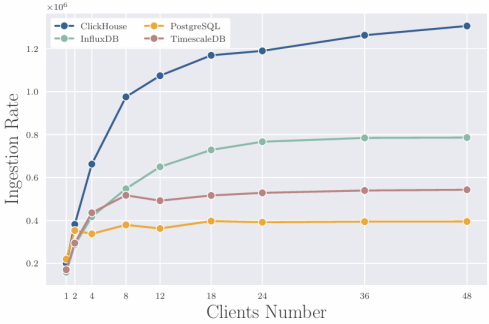


Figure 3. Ingestion rate as a function of the number of concurrent clients.

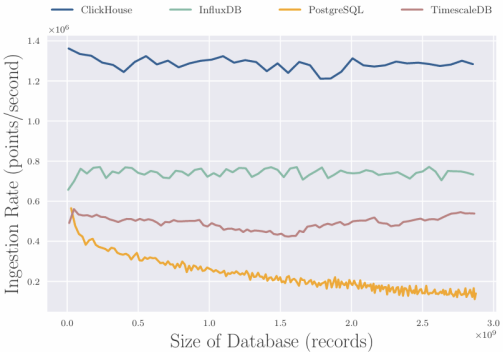
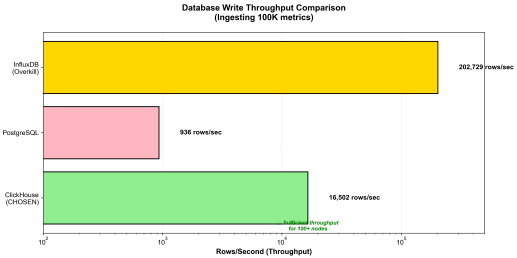
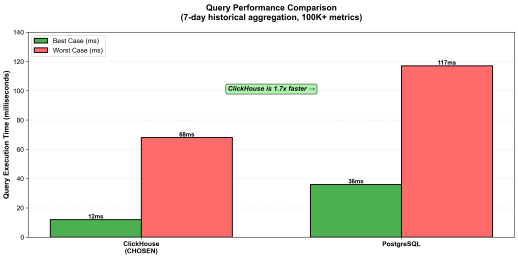


Figure 5. Ingestion rate as a function of database size.

💡 This pair extends the argument beyond our machine: Figure 3 favors ClickHouse for many concurrent writers, and Figure 5 favors ClickHouse as retained history grows larger.

Benchmark Results





16,502
rows/sec write throughput

12–68 ms
query latency range

Low ops
vs. specialized TSDB stack

💡 This pair shows the local result clearly: our setup already gives acceptable writes and very fast analytical queries, which is why ClickHouse fits the dashboard workload.

Ensembled Results, Tradeoff, and Choice

Ensembled Results Analysis

- Our benchmarks show **fast queries** and **sufficient write throughput** for the project
- The paper adds evidence for **better concurrency scaling** and **better stability as database size grows**
- Together, the local and external results tell a consistent story in favor of ClickHouse

Tradeoff Acknowledgment

- InfluxDB is still strong on ingestion and has a light memory footprint
- The paper presents it as competitive, not weak
- We still choose ClickHouse because our system is query-heavy and grows into a larger historical analytics workload

ClickHousen because it aligns better with our mix of real-time dashboards, historical queries, concurrent writers, and long-term scaling.

Impact and Scalability Path

Current Value

- Real-time energy visibility
- Per-node and per-app analysis
- Early detection of unhealthy nodes
- Historical views for optimization

Scalability Path

- **Today**: proof of concept on a small node set
- **Next**: tens to hundreds of nodes, same API
- **Later**: ClickHouse sharding + replicated backends
- No architecture redesign required

We built a production-style distributed monitoring architecture based on autonomy, eventual consistency, adaptive health checking, and analytical storage.

Demo Video

Demo

Watch Video

Click to open

Thank You

Questions?

Distributed Energy Monitoring Platform